

Phase 1 - Dual Secret Storage

Step 1.1 — Create SSM Parameter Store Entries

Console Method

1. Go to **AWS Systems Manager** → **Parameter Store**
2. Click **Create parameter** for each:

Name	Type	Value
/lab/db/endpoint	SecureString	Your RDS endpoint
/lab/db/port	String	3306
/lab/db/name	String	labdb

CLI Method

```
aws ssm put-parameter \  
  --name /lab/db/endpoint \  
  --value "<YOUR_RDS_ENDPOINT>" \  
  --type SecureString
```

```
aws ssm put-parameter \  
  --name /lab/db/port \  
  --value "3306" \  
  --type String
```

```
aws ssm put-parameter \  
  --name /lab/db/name \  
  --value "labdb" \  
  --type String
```

Verification

```
aws ssm get-parameters \  
  --names /lab/db/endpoint /lab/db/port /lab/db/name \  
  --with-decryption
```



Expected: All three parameters returned with correct values.

Step 1.2 — Create Secrets Manager Secret

Console Method

1. Go to **Secrets Manager** → **Store a new secret**
2. Choose **Other type of secret**
3. Add key/value pairs:
 - `username` = `admin`
 - `password` = `\<YOUR_PASSWORD\>`
 - `host` = `\<YOUR_RDS_ENDPOINT\>`
 - `port` = `3306`
4. Secret name: `lab/rds/mysql`

CLI Method

```
aws secretsmanager create-secret \  
  --name lab/rds/mysql \  
  --secret-string '{"username":"admin","password":"<YOUR_PASS  
WORD>","host":"<YOUR_RDS_ENDPOINT>","port":"3306"}'
```

Verification

```
aws secretsmanager get-secret-value \  
  --secret-id lab/rds/mysql
```



Expected: JSON output with all four fields present.

Step 1.3 — Update IAM Role Permissions

Your EC2 instance role needs permissions for both systems.

SSM Parameter Store Policy (Add as inline policy)

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "ReadLabParameters",  
      "Effect": "Allow",  
      "Action": [  
        "ssm:GetParameter",  
        "ssm:GetParameters"  
      ],  
      "Resource": "arn:aws:ssm:us-east-2:<ACCOUNT_ID>:parameter/lab/db/*"  
    }  
  ]  
}
```

Secrets Manager Policy (Add as inline policy)

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "SecretsManagerAccess",  
      "Effect": "Allow",  
      "Action": [  
        "secretsmanager:GetSecretValue",  
        "secretsmanager:DescribeSecret",  
        "secretsmanager:ListSecretValues"  
      ],  
      "Resource": "*"
```

```
{
  "Sid": "ReadSpecificSecret",
  "Effect": "Allow",
  "Action": ["secretsmanager:GetSecretValue"],
  "Resource": "arn:aws:secretsmanager:us-east-2:<ACCOUNT_ID>:secret:lab/rds/mysql*"
}
```



Note the wildcard: Secrets Manager appends random characters to secret ARNs. The `\*` at the end is required.

Step 1.4 — Verify EC2 Can Read Both Systems

SSH into your EC2 instance and run:

```
# Test SSM
aws ssm get-parameter --name /lab/db/endpoint

# Test Secrets Manager
aws secretsmanager get-secret-value --secret-id lab/rds/mysql
```



Expected: Both commands succeed with no `AccessDeniedException`.

Common Issues — Phase 1

▼ Issue: `AccessDeniedException` on SSM

Symptom: An error occurred (`AccessDeniedException`) when calling the `GetParameter` operation

Root Cause: Missing `ssm:GetParameter` or `ssm:GetParameters` action in IAM policy

Fix:

1. IAM Console → Roles → Your EC2 role
2. Add permissions → Create inline policy
3. Paste the SSM policy above
4. Retry from EC2

▼ Issue: `AccessDeniedException` on Secrets Manager

Symptom: An error occurred (`AccessDeniedException`) when calling the `GetSecretValue` operation

Possible Causes:

1. **ARN mismatch:** Run `aws secretsmanager list-secrets` and compare ARN exactly
2. **Custom KMS key:** Check secret's encryption key; add `kms:Decrypt` if custom
3. **Missing wildcard:** Secret ARN has random suffix; policy must end with `\*`

Diagnostic Command:

```
aws secretsmanager list-secrets --query "SecretList[?Name=
='lab/rds/mysql'].ARN" --output text
```

▼ Issue: Trust Relationship Missing

Symptom: EC2 can't assume the role at all

Fix: Ensure role's trust policy includes:

```
{
  "Service": "
```

Phase 1 Checklist

- ✓ Parameter Store: `/lab/db/endpoint` created
- ✓ Parameter Store: `/lab/db/port` created

- ✓ Parameter Store: `/lab/db/name` created
- ✓ Secrets Manager: `lab/rds/mysql` created with JSON payload
- ✓ IAM: SSM read policy attached to EC2 role
- ✓ IAM: Secrets Manager read policy attached to EC2 role
- ✓ Verification: EC2 can read SSM parameters
- ✓ Verification: EC2 can read Secrets Manager secret

Phase 1 Execution Log (January 2026)



Status: COMPLETE — All Phase 1 steps executed successfully after detecting and remediating sabotage.

Actual Values Used

Parameter	Value
<code>/lab/db/endpoint</code>	<code>helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com</code>
<code>/lab/db/port</code>	<code>3306</code>
<code>/lab/db/name</code>	<code>labdb</code>
Secret ID	<code>lab/rds/mysql</code>
Region	<code>us-east-2</code>

Sabotage Detected & Remediated

Group lead used shared IAM credentials to inject these configuration errors:

Attack Type	Resource	Poisoned Value	Correct Value	Detection
Character Substitution	SSM endpoint	<code>helga-rds01...</code> (numeral <code>1</code>)	<code>helga-rds01...</code> (letter <code>l</code>)	<code>Unknown MySQL server host</code> in logs
Prefix Injection	SSM db name	<code>t_labdb</code>	<code>labdb</code>	<code>Unknown database</code> error

Attack Type	Resource	Poisoned Value	Correct Value	Detection
Character Substitution	Secret username	<code>adm1n</code> (numeral 1)	<code>admin</code> (letter i)	Access denied for user 'adm1n' in CloudWatch

Key Insight: The `helga` typo (numeral 1 vs letter l) is visually almost identical. Always verify configuration values programmatically, never visually.

Fix Commands Executed

```
# Fix SSM endpoint
MSYS_NO_PATHCONV=1 aws ssm put-parameter \
  --name /lab/db/endpoint \
  --value "helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com" \
  --type SecureString --overwrite --region us-east-2

# Fix SSM db name
MSYS_NO_PATHCONV=1 aws ssm put-parameter \
  --name /lab/db/name \
  --value "labdb" \
  --type String --overwrite --region us-east-2

# Fix secret username
MSYS_NO_PATHCONV=1 aws secretsmanager update-secret \
  --secret-id lab/rds/mysql \
  --secret-string '{"username":"admin","password":"<PASSWORD>","host":"helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com","port":"3306"}' \
  --region us-east-2
```

Verification Results

All 5 checks passed after fixes:

-  RDS Status: available
-  SSM endpoint matches RDS
-  Secret host matches RDS
-  SSM port: 3306
-  SSM db name: labdb

App endpoint test: `/init` returned `"Initialized t_labdb + notes table"` confirming DB connection working.

Here's a shareable step-by-step guide for creating Lab 1B CLI credentials:

Lab 1B CLI Credentials Setup (Shareable Guide)

Prerequisites

- AWS Console access with IAM permissions
 - AWS CLI installed on your machine
-

Step 1: Log into AWS Console

1. Go to console.aws.amazon.com
 2. Navigate to **IAM** → **Users** → **Create user**
-

Step 2: Create the User

1. **User name:** `lab1b-yourname` (e.g., `lab1b-willie`)
 2. **Do NOT** check "Provide user access to the AWS Management Console"
 3. Click **Next**
-

Step 3: Attach Permissions

1. Select **Attach policies directly**
2. Click **Create policy** (opens new tab)
3. Select **JSON** tab

4. Paste this policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Lab1B0perations",
      "Effect": "Allow",
      "Action": [
        "ssm:GetParameter",
        "ssm:GetParameters",
        "ssm:PutParameter",
        "secretsmanager:GetSecretValue",
        "secretsmanager:CreateSecret",
        "secretsmanager:UpdateSecret",
        "secretsmanager:ListSecrets",
        "logs:*",
        "cloudwatch:PutMetricAlarm",
        "cloudwatch:DescribeAlarms",
        "cloudwatch:DescribeAlarmHistory",
        "cloudwatch:SetAlarmState",
        "ec2:DescribeInstances",
        "ec2:DescribeSecurityGroups",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:RevokeSecurityGroupIngress",
        "rds:DescribeDBInstances",
        "rds:StopDBInstance",
        "rds:StartDBInstance",
        "sts:GetCallerIdentity"
      ],
      "Resource": "*"
    }
  ]
}
```

1. Click **Next**
 2. **Policy name:** Lab1B-Access
 3. Click **Create policy**
 4. Go back to the user creation tab → **Refresh** → Search for Lab1B-Access → **Check it**
 5. Click **Next** → **Create user**
-

Step 4: Create Access Keys

1. Click on your new user
 2. Go to **Security credentials** tab
 3. Scroll to **Access keys** → **Create access key**
 4. Select **Command Line Interface (CLI)**
 5. Check the confirmation box → **Next** → **Create access key**
 6. **COPY BOTH VALUES NOW** (you can't see the secret again):
 - Access key ID: AKIA...
 - Secret access key: wJalr...
-

Step 5: Configure Your CLI

Open terminal and run:

```
aws configure --profile lab1b
```

Enter when prompted:

```
AWS Access Key ID: [paste your access key]
AWS Secret Access Key: [paste your secret key]
Default region name: us-east-2
Default output format: json
```

Step 6: Test It

```
aws sts get-caller-identity --profile lab1b
```

You should see your user ARN.

Step 7: Use for All Lab Commands

Option A: Add `--profile lab1b` to every command:

```
aws ssm get-parameter --name /lab/db/endpoint --profile lab1b
```

Option B: Set as default for your terminal session:

```
export AWS_PROFILE=lab1b
```

Done!

You now have least-privilege credentials for Lab 1B operations.

Want me to add this as a page in your Lab 1B guide so you can share the link with your group?

Ha! Alright, let's red team this. Here's everything your group lead can do with those permissions to make your life interesting:

Red Team Playbook: Lab 1B Attack Vectors

1. Credential Poisoning (Secrets Manager)

Attack	What They Do	Error You'll See
Wrong	<code>UpdateSecret</code> with bad password	<code>Access denied for user 'admin'</code>

Attack	What They Do	Error You'll See
password		
Wrong host	Change <code>host</code> field to fake endpoint	<code>Unknown MySQL server host</code>
Wrong port	Change port to 3307 or something	<code>Connection refused</code>
Malformed JSON	Break the JSON structure	App crashes on parse
Decoy secrets	Create <code>lab/rds/mysql-backup</code> or similar	You waste time checking wrong secret

Detection:

```
# Check secret content
aws secretsmanager get-secret-value --secret-id lab/rds/mysql
--query "SecretString" --output text | jq .

# Check for recent modifications
aws secretsmanager describe-secret --secret-id lab/rds/mysql
--query "LastChangedDate"
```

2. Parameter Store Corruption

Attack	What They Do	Effect
Wrong endpoint	Change <code>/lab/db/endpoint</code> to garbage	<code>Unknown host</code> errors
Wrong port	Change <code>/lab/db/port</code> to <code>22</code>	Connection refused or SSH banner
Invisible whitespace	Add space/newline to values	Subtle connection failures

Detection:

```
# Verify all params (pipe to xxd to catch invisible chars)
aws ssm get-parameters --names /lab/db/endpoint /lab/db/port
```

```
/lab/db/name --with-decryption --query "Parameters[*].[Name,Value]" --output text | xxd
```

3. Security Group Chaos (EC2)

Attack	What They Do	Error You'll See
Revoke RDS rule	Remove EC2 → RDS ingress	Connection timed out
Wrong source SG	Re-add rule with different source group	Still times out
Port mismatch	Allow port 3307 instead of 3306	Times out
Red herring rules	Add suspicious-looking rules that don't break anything	You waste time investigating

Detection:

```
# Dump all rules on RDS SG
aws ec2 describe-security-groups --group-ids <RDS_SG_ID> --query "SecurityGroups[0].IpPermissions"

# Verify specific rule exists
aws ec2 describe-security-groups --group-ids <RDS_SG_ID> \
  --query "SecurityGroups[0].IpPermissions[?FromPort==\`3306\`]"
```

4. RDS Manipulation

Attack	What They Do	Error You'll See
Stop instance	StopDBInstance	Connection refused
Stop/start cycling	Intermittent availability	Random failures, hard to reproduce

Detection:

```
# Check RDS status
aws rds describe-db-instances --db-instance-identifier lab-my
```

```
sql --query "DBInstances[0].DBInstanceStatus"

# Check for recent events
aws rds describe-events --source-identifier lab-mysql --source-type db-instance --duration 60
```

5. CloudWatch Log Manipulation

Attack	What They Do	Effect
Delete log group	Remove <code>/aws/ec2/lab-rds-app</code>	No logs to diagnose from
Inject fake ERRORS	Add misleading log entries	You chase ghosts
Change metric filter	Alter filter pattern to <code>ERRORZ</code>	Alarm never fires

Detection:

```
# Verify log group exists
aws logs describe-log-groups --log-group-name-prefix /aws/ec2/lab-rds-app

# Check metric filters
aws logs describe-metric-filters --log-group-name /aws/ec2/lab-rds-app
```

6. CloudWatch Alarm Manipulation

Attack	What They Do	Effect
Delete alarm	Remove the alarm entirely	No alerting
Raise threshold	Set threshold to 9999	Never triggers
Manual state override	<code>SetAlarmState</code> to OK	Hides real incidents
Decoy alarms	Create similar-named alarms	Confusion

Detection:

```
# List all alarms
aws cloudwatch describe-alarms --alarm-name-prefix lab

# Check alarm state AND configuration
aws cloudwatch describe-alarms --alarm-names lab-db-connection-failure-alarm
```

7. Multi-Vector Combinations (The Nasty Ones)

Combo	What They Do	Why It's Evil
Wrong password + alarm suppression	Bad creds + <code>SetAlarmState</code> <code>OK</code>	App fails but alarm says everything's fine
SG revoke + fake ERROR logs	Block traffic + inject misleading errors	You diagnose the wrong thing
Parameter poison + correct secret	Wrong endpoint in SSM, correct in Secrets Manager	Error message doesn't match what you find in secrets
Stop RDS + delete log group	Kill DB + remove evidence	Blind incident response

Your Defense Checklist

Before every incident response, run this verification sweep:

```
#!/bin/bash
echo "=== 1. SECRETS MANAGER ==="
aws secretsmanager get-secret-value --secret-id lab/rds/mysql \
--query "SecretString" --output text | jq .
aws secretsmanager describe-secret --secret-id lab/rds/mysql \
--query "LastChangedDate"

echo "=== 2. PARAMETER STORE ==="
aws ssm get-parameters --names /lab/db/endpoint /lab/db/port \
/lab/db/name --with-decryption
```

```

echo "=== 3. RDS STATUS ==="
aws rds describe-db-instances --db-instance-identifier lab-my
sql --query "DBInstances[0].{Status:DBInstanceStatus,Endpoint:Endpoint.Address}"

echo "=== 4. SECURITY GROUP RULES ==="
aws ec2 describe-security-groups --group-ids <RDS_SG_ID> --query "SecurityGroups[0].IpPermissions[?FromPort==\`3306\`]"

echo "=== 5. LOG GROUP ==="
aws logs describe-log-groups --log-group-name-prefix /aws/ec2/lab-rds-app

echo "=== 6. METRIC FILTER ==="
aws logs describe-metric-filters --log-group-name /aws/ec2/lab-rds-app

echo "=== 7. ALARM STATE ==="
aws cloudwatch describe-alarms --alarm-names lab-db-connection-failure-alarm --query "MetricAlarms[0].{State:StateValue,Threshold:Threshold}"

```

Now you're ready for whatever chaos they throw at you. Good luck! 🍀

Here you go:

```

aws rds describe-db-instances --db-instance-identifier lab-my
sql --query "DBInstances[0].VpcSecurityGroups[*].VpcSecurityGroupId" --output text

```

If you don't know the exact DB instance identifier, run this first to list all RDS instances:

```

aws rds describe-db-instances --query "DBInstances[*].[DBInstanceIdentifier,VpcSecurityGroups[*].VpcSecurityGroupId]" --ou

```

```
tput table
```

Two separate problems here:

Problem 1: Git Bash Path Conversion (Windows)

Git Bash is converting `/lab/...` and `/aws/...` to Windows paths. That's why you see `C:/Program Files/Git/lab/db/endpoint`.

Fix: Prefix commands with `MSYS_NO_PATHCONV=1` or double the leading slash:

```
# Option A: Disable path conversion for the command
MSYS_NO_PATHCONV=1 aws ssm get-parameters --names /lab/db/end
point /lab/db/port /lab/db/name --with-decryption
```

```
# Option B: Double the leading slash
aws ssm get-parameters --names //lab/db/endpoint //lab/db/por
t //lab/db/name --with-decryption
```

```
# Option C: Use quotes with double slash for log groups
aws logs describe-log-groups --log-group-name-prefix "//aws/e
c2/lab-rds-app"
```

Problem 2: Resources Don't Exist (Yet or Anymore)

Before you can run the defense checklist, you need to **discover what actually exists**. Run these:

```
# 1. List ALL RDS instances (find your actual DB identifier)
aws rds describe-db-instances --query "DBInstances[*].[DBInst
anceIdentifier,DBInstanceStatus,Endpoint.Address]" --output t
able
```

```
# 2. List ALL secrets
aws secretsmanager list-secrets --query "SecretList[*].[Name,
ARN]" --output table
```

```
# 3. List ALL SSM parameters under /lab (with path fix)
MSYS_NO_PATHCONV=1 aws ssm describe-parameters --parameter-filters "Key=Path,Values=/lab" --query "Parameters[*].[Name,Type]" --output table

# 4. List ALL log groups
aws logs describe-log-groups --query "logGroups[*].[logGroupName]" --output table

# 5. List ALL CloudWatch alarms
aws cloudwatch describe-alarms --query "MetricAlarms[*].[AlarmName,StateValue]" --output table
```

What To Do Next

Once you run those discovery commands, you'll know:

If you see...	Then...
No RDS instances at all	Lab infrastructure isn't deployed yet, or was destroyed
RDS with different name	Update your commands to use the correct identifier
No secrets	Need to create them per Phase 1
Secrets with different names	Your group lead may have renamed/recreated them
No log groups	CloudWatch Agent isn't running or log group was deleted

Run those discovery commands and share what you find — then we'll know if your group lead nuked everything or if the lab just needs to be set up from scratch. 🔍

Got it. Here's the diagnosis:

Current State Summary

Resource	Status	What Exists
RDS	✗ None	No DB instances in <code>us-east-1</code>
Secrets Manager	⚠ Different name	<code>helga/rds/mysqlv2</code> (not <code>lab/rds/mysql</code>)

Resource	Status	What Exists
SSM Parameters	✗ None	No <code>/lab/*</code> parameters
Log Groups	⚠ Different names	<code>/helga/ec2/user-data</code> , <code>/helga/notes-app</code>
CloudWatch Alarms	✗ None	No alarms

Bottom line: Lab 1B infrastructure doesn't exist. You have **Lab 1C (helga)** resources, but Lab 1B expects `lab/` naming.

Region Check First

Your secret is in **us-east-1**, but your Lab 1C Terraform was in **us-east-2**. Run this to confirm your RDS location:

```
# Check us-east-2 for RDS
aws rds describe-db-instances --region us-east-2 --query "DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus,Endpoint.Address]" --output table
```

Your Options

Option A: Use Existing Helga Infrastructure (Fastest)

Adapt your Lab 1B practice to your existing resources:

```
# 1. Check what's in your existing secret
aws secretsmanager get-secret-value --secret-id helga/rds/mysqlv2 --query "SecretString" --output text | jq .
```

```
# 2. Check for helga SSM parameters
MSYS_NO_PATHCONV=1 aws ssm describe-parameters --parameter-filters "Key=Path,Values=/helga" --query "Parameters[*].[Name,Type]" --output table
```

```
# 3. Check us-east-2 for your RDS
aws rds describe-db-instances --region us-east-2 --query "DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus,Endpoint.Address]" --output table
```

```
nstances[*].[DBInstanceIdentifier,DBInstanceStatus,Endpoint.Address]" --output table
```

Then update your defense checklist to use `helga` names instead of `lab` names.

Option B: Create Fresh Lab 1B Resources (Full Setup)

If you want isolated Lab 1B resources with the `lab/` naming:

Step 1: Create SSM Parameters

```
# First, get your RDS endpoint from us-east-2
RDS_ENDPOINT=$(aws rds describe-db-instances --region us-east-2 --query "DBInstances[0].Endpoint.Address" --output text)
echo "RDS Endpoint: $RDS_ENDPOINT"

# Then create the parameters (use MSYS fix)
MSYS_NO_PATHCONV=1 aws ssm put-parameter --name /lab/db/endpoint --value "$RDS_ENDPOINT" --type SecureString
MSYS_NO_PATHCONV=1 aws ssm put-parameter --name /lab/db/port --value "3306" --type String
MSYS_NO_PATHCONV=1 aws ssm put-parameter --name /lab/db/name --value "labdb" --type String
```

Step 2: Create Lab Secret

```
# Get password from existing secret
EXISTING_SECRET=$(aws secretsmanager get-secret-value --secret-id helga/rds/mysqlv2 --query "SecretString" --output text)
echo "$EXISTING_SECRET" | jq .

# Create new lab secret with same creds
MSYS_NO_PATHCONV=1 aws secretsmanager create-secret \
  --name lab/rds/mysql \
  --secret-string "{\"username\":\"admin\", \"password\":\"<FROM_ABOVE>\", \"host\":\"$RDS_ENDPOINT\", \"port\":\"3306\"}"
```

```
MSYS_NO_PATHCONV=1 aws secretsmanager create-secret \
--name lab/rds/mysql \
--secret-string "{\"username\":\"admin\", \"password\":\"i8
$2>Iu]dQ0h60lzzRG#iHJS1mg7\", \"host\":\"helga-rds1ab2a.chkce0
2amfxr.us-east-1.rds.amazonaws.com\", \"port\":\"3306\"}"
```

⚡ Recommended Next Step

Run the region check first — I suspect your RDS is in **us-east-2** while you're querying **us-east-1**:

```
aws rds describe-db-instances --region us-east-2 --query "DBI
nstances[*].[DBInstanceIdentifier,DBInstanceStatus,Endpoint.A
ddress]" --output table
```

Share that output and I'll give you the exact commands to get everything aligned.

🚨 **Your group lead has already started the chaos!** Look at these mismatches:

🔍 Mismatch Analysis

Resource	Field	Current Value	Should Be	Issue
SSM	/lab/db/endpoint	helga-rds01...	helga-rds01...	Typo: numeral 1 instead of letter l
SSM	/lab/db/name	t_labdb	labdb	Extra t_ prefix
Secret	host	lab-mysql...	helga-rds01...	Points to non-existent RDS
Actual RDS	identifier	helga-rds01	—	This is your real DB

Fix Commands

Run these to align everything with your actual RDS (`helga-rds01`):

Fix 1: Correct the SSM endpoint (typo fix)

```
MSYS_NO_PATHCONV=1 aws ssm put-parameter \  
  --name /lab/db/endpoint \  
  --value "
```

Fix 2: Correct the SSM db name

```
MSYS_NO_PATHCONV=1 aws ssm put-parameter \  
  --name /lab/db/name \  
  --value "labdb" \  
  --type String \  
  --overwrite \  
  --region us-east-2
```

Fix 3: Update the secret to point to correct RDS

```
MSYS_NO_PATHCONV=1 aws secretsmanager update-secret \  
  --secret-id lab/rds/mysql \  
  --secret-string '{"username":"admin","password":"i8$l>Iu]dN  
0h60lzzRG#iHJR1mg7","engine":"mysql","host":'
```

Verify fixes

```
REGION="us-east-2"
```

```
echo "=== SSM ==="  
MSYS_NO_PATHCONV=1 aws ssm get-parameters --names /lab/db/end  
point /lab/db/name --with-decryption --region $REGION --query  
"Parameters[*].[Name,Value]" --output table
```

```
echo "=== SECRET ==="
MSYS_NO_PATHCONV=1 aws secretsmanager get-secret-value --secret-id lab/rds/mysql --region $REGION --query "SecretString" -
-output text | jq '.host'
```

Pro tip: That `helga` typo (numeral 1 vs letter l) is a classic sabotage move — visually almost identical but causes `Unknown host` errors. Now you know what to look for! 🙄

Here's your full verification runbook:

✓ Lab 1B Verification Checklist

Step 1: Verify Config Alignment (run this first)

```
REGION="us-east-2"
```

```
echo "=== ACTUAL RDS ==="
aws rds describe-db-instances --db-instance-identifier helga-
rds01 --region $REGION --query "DBInstances[0].{Status:DBInst
anceStatus,Endpoint:Endpoint.Address}" --output table
```

```
echo ""
echo "=== SSM PARAMETERS ==="
MSYS_NO_PATHCONV=1 aws ssm get-parameters --names /lab/db/end
point /lab/db/port /lab/db/name --with-decryption --region $R
EGION --query "Parameters[*].[Name,Value]" --output table
```

```
echo ""
echo "=== SECRETS MANAGER ==="
MSYS_NO_PATHCONV=1 aws secretsmanager get-secret-value --secr
et-id lab/rds/mysql --region $REGION --query "SecretString" -
-output text | jq '{host,port,username}'
```

Expected output:

Source	Field	Should Match
RDS	Endpoint	<code>'helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com'</code>
SSM	<code>/lab/db/endpoint</code>	Same as RDS endpoint
Secret	<code>host</code>	Same as RDS endpoint
SSM	<code>/lab/db/port</code>	<code>3306</code>
Secret	<code>port</code>	<code>3306</code>

Step 2: Test Database Connectivity (from EC2)

If you have the Flask app or `mysql` client on your EC2:

```
# Option A: Quick connectivity test with nc (netcat)
nc -zv helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com 3306

# Option B: MySQL client test
mysql -h helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com
-u admin -p -e "SELECT 1;"
```

Expected:

- `nc`: `Connection to ... 3306 port \[tcp/mysql\] succeeded!`
- `mysql`: Returns `1` after entering password

Step 3: Verify Security Group Allows Traffic

```
REGION="us-east-2"

# Get the RDS security group
RDS_SG=$(aws rds describe-db-instances --db-instance-identifier helga-rds01 --region $REGION --query "DBInstances[0].VpcSecurityGroups[0].VpcSecurityGroupId" --output text)

echo "RDS Security Group: $RDS_SG"
```

```
# Check for port 3306 rule
aws ec2 describe-security-groups --group-ids $RDS_SG --region
$REGION --query "SecurityGroups[0].IpPermissions[?FromPort==
\`3306\`]" --output table
```

Expected: At least one rule allowing port 3306 from your EC2's security group or CIDR.

Step 4: Full End-to-End Smoke Test

Run this single script to check everything at once:

```
#!/bin/bash
REGION="us-east-2"
RDS_ID="helga-rds01"
PASS=0
FAIL=0

echo "🔍 Lab 1B Verification Starting..."
echo "===== "

# 1. RDS Status
RDS_STATUS=$(aws rds describe-db-instances --db-instance-iden
tifier $RDS_ID --region $REGION --query "DBInstances[0].DBIns
tanceStatus" --output text 2>/dev/null)
if [ "$RDS_STATUS" == "available" ]; then
    echo "✅ RDS Status: $RDS_STATUS"
    ((PASS++))
else
    echo "❌ RDS Status: $RDS_STATUS (expected: available)"
    ((FAIL++))
fi

# 2. SSM Endpoint matches RDS
RDS_ENDPOINT=$(aws rds describe-db-instances --db-instance-id
```

```

entifier $RDS_ID --region $REGION --query "DBInstances[0].End
point.Address" --output text)
SSM_ENDPOINT=$(MSYS_NO_PATHCONV=1 aws ssm get-parameter --nam
e /lab/db/endpoint --with-decryption --region $REGION --query
"Parameter.Value" --output text)
if [ "$RDS_ENDPOINT" == "$SSM_ENDPOINT" ]; then
    echo "✅ SSM endpoint matches RDS"
    ((PASS++))
else
    echo "❌ SSM endpoint MISMATCH"
    echo "    RDS: $RDS_ENDPOINT"
    echo "    SSM: $SSM_ENDPOINT"
    ((FAIL++))
fi

```

3. Secret host matches RDS

```

SECRET_HOST=$(MSYS_NO_PATHCONV=1 aws secretsmanager get-secre
t-value --secret-id lab/rds/mysql --region $REGION --query "S
ecretString" --output text | jq -r '.host')
if [ "$RDS_ENDPOINT" == "$SECRET_HOST" ]; then
    echo "✅ Secret host matches RDS"
    ((PASS++))
else
    echo "❌ Secret host MISMATCH"
    echo "    RDS: $RDS_ENDPOINT"
    echo "    Secret: $SECRET_HOST"
    ((FAIL++))
fi

```

4. SSM port is 3306

```

SSM_PORT=$(MSYS_NO_PATHCONV=1 aws ssm get-parameter --name /l
ab/db/port --region $REGION --query "Parameter.Value" --outpu
t text)
if [ "$SSM_PORT" == "3306" ]; then
    echo "✅ SSM port: 3306"
    ((PASS++))

```

```

else
    echo "❌ SSM port: $SSM_PORT (expected: 3306)"
    ((FAIL++))
fi

# 5. SSM db name is labdb
SSM_NAME=$(MSYS_NO_PATHCONV=1 aws ssm get-parameter --name /lab/db/name --region $REGION --query "Parameter.Value" --output text)
if [ "$SSM_NAME" == "labdb" ]; then
    echo "✅ SSM db name: labdb"
    ((PASS++))
else
    echo "❌ SSM db name: $SSM_NAME (expected: labdb)"
    ((FAIL++))
fi

echo "====="
echo "Results: $PASS passed, $FAIL failed"

if [ $FAIL -eq 0 ]; then
    echo "🎉 All checks passed! Lab 1B is ready."
else
    echo "⚠️ Fix the failures above before proceeding."
fi

```

Run Step 1 first to see current state, then run Step 4 for the full automated check. Let me know what you get! 🚀

Smart move deactivating those keys! 😊

Here's **Phase 2: Observability Setup** — getting your app logs into CloudWatch so you can create alarms.

Phase 2 Overview

Step	What You're Doing	Where
2.1	Redirect Flask logs to a file	EC2
2.2	Add <code>ERROR</code> keyword to app logging	EC2
2.3	Install & configure CloudWatch Agent	EC2
2.4	Add IAM permissions for CloudWatch Logs	AWS Console
2.5	Verify log group appears	CLI

Step 2.1 — Redirect Flask Logs to File

SSH into your EC2 and run:

```
# Create log directory
sudo mkdir -p /var/log/rdsapp

# Edit the systemd service
sudo nano /etc/systemd/system/rdsapp.service
```

Add these two lines under the `[Service]` section:

```
StandardOutput=append:/var/log/rdsapp/app.log
StandardError=append:/var/log/rdsapp/app.log
```

Then reload and restart:

```
sudo systemctl daemon-reload
sudo systemctl restart rdsapp

# Verify logs are flowing
sudo tail -f /var/log/rdsapp/app.log
```

Expected: Flask startup messages appear. Press `Ctrl+C` to exit.

738-772-514

Step 2.2 — Add ERROR Logging to `app.py`

This is **critical** — without `ERROR` in your logs, the alarm won't trigger.

```
sudo nano /opt/rdsapp/
```

Find the `get_conn()` function and make sure it has this line in the `except` block:

```
print(f"ERROR: Database connection failed - {e}")
```

Then restart:

```
sudo systemctl restart rdsapp
```

Ready to run Step 2.1? Once you've got logs flowing to the file, let me know and we'll continue with the CloudWatch Agent setup.

Step 2.3 — Install & Configure CloudWatch Agent

SSH into your EC2 and run these commands:

Install the Agent

```
sudo yum install -y amazon-cloudwatch-agent
```

Create the Config File

```
sudo nano /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
```

Paste this configuration:

```
{
  "agent": {
    "run_as_user": "root"
  },

```

```

"logs": {
  "logs_collected": {
    "files": {
      "collect_list": [
        {
          "file_path": "/var/log/rdsapp/app.log",
          "log_group_name": "/aws/ec2/lab-rds-app",
          "log_stream_name": "{instance_id}",
          "timezone": "UTC"
        }
      ]
    }
  }
}

```

Start the Agent

```

sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-a
gent-ctl \
  -a fetch-config \
  -m ec2 \
  -c

```

Expected: Status shows `"status": "running"`

Step 2.4 — Add IAM Permissions

Your EC2 instance role needs CloudWatch Logs permissions. Add this inline policy:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {

```

```
"Sid": "CloudWatchLogs",
"Effect": "Allow",
"Action": [
    "logs:CreateLogGroup",
    "logs:CreateLogStream",
    "logs:PutLogEvents",
    "logs:DescribeLogStreams"
],
"Resource": "arn:aws:logs:us-east-2:919113286081:log-group:/aws/ec2/lab-rds-app:*"
}
```

Step 2.5 — Verify Log Group Appears

Wait ~1 minute, then check:

```
MSYS_NO_PATHCONV=1 aws logs describe-log-groups \
  --log-group-name-prefix /aws/ec2/lab-rds-app \
  --region us-east-2
```

Expected: You see the `/aws/ec2/lab-rds-app` log group.

Then verify logs are flowing:

```
MSYS_NO_PATHCONV=1 aws logs tail /aws/ec2/lab-rds-app --region
us-east-2 --follow
```

Let me know once logs appear in CloudWatch and we'll move to **Phase 3: Metric Filters & Alarms!** 🚀

